FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int p, r, i, j, k;

    printf("Enter number of processes: \n");
    scanf("%d", &p);

    printf("Enter number of resource types: \n");
    scanf("%d", &r);

    int alloc[p][r], max[p][r], need[p][r], avail[r];
    int finish[p], safeSeq[p];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            scanf("%d", &max[i][j]);

```

Test Cases 2

Run Sample Cases

Run All Cases

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```c
    printf("Enter Available Resources: \n");
    for(i = 0; i < r; i++)
        scanf("%d", &avail[i]);

    for(i = 0; i < p; i++)
        for(j = 0; j < r; j++)
            need[i][j] = max[i][j] - alloc[i][j];

    for(i = 0; i < p; i++)
        finish[i] = 0;

    int count = 0;

    while(count < p) {
        int found = 0;

        for(i = 0; i < p; i++) {
            if(finish[i] == 0) {
                int flag = 0;

                for(j = 0; j < r; j++) {
                    if(need[i][j] > avail[j]) {
                        flag = 1;
                        break;
```

Test Cases 2

Run Sample Cases

Run All Cases

FACE Prep

Operating System - Coding | REC | 3 Feb

Course Progress

Go Back

Back To Practice

Q1

Save

Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```c

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("System is NOT in a safe state (Deadlock may occur).\n");
            return 0;
        }
    }

    printf("System is in a SAFE STATE.\n");
    printf("Safe Sequence: ");

    for(i = 0; i < p; i++)
        printf("P%d ", safeSeq[i]);

    printf("\n");

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Back To Practice

Q1

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Custom Test

Success

Input :

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of

Input :

```
3
3
0 1 0
2 0 0
3 0 3
3 2 2
2 1 1
4 3 3
0 0 0
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is NOT in a safe state (Deadlock may occur).
```

Back To Practice

Q1

Save

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Input :

```
3
2
1 0
0 1
1 1
2 1
1 2
3 3
1 1
```

Expected Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is in a SAFE STATE.
Safe Sequence: P0 P1 P2
```

Output :

```
Enter number of processes:
Enter number of resource types:
Enter Allocation Matrix:
Enter Max Matrix:
Enter Available Resources:
System is in a SAFE STATE.
Safe Sequence: P0 P1 P2
```